# SmartChem: Backend Preparation & Presentation Guide

This guide is designed to prepare you for a technical presentation and Q&A (viva) regarding the SmartChem backend. It covers the system architecture, API design, database schemas, and deep dives into common technical questions.

---

## 1. Backend Overview

**Purpose:** The backend serves as the bridge between the user interface, the secure database, and the heavy Machine Learning (ML) inference engine.
**Key Responsibilities:**

- Manage user authentication and sessions securely.
- Handle CRUD operations for projects and saved molecules.
- Offload intense Machine Learning tasks (generation and optimization) asynchronously so the API remains highly responsive.
- Perform fast, synchronous cheminformatics computations (ADMET, Lipinski, 3D structures) using RDKit.
- Integrate external APIs (e.g., Groq for the AI chat assistant).

## 2. Architecture & System Design

**Overall Architecture:** Monolithic API with an Asynchronous Producer-Consumer Worker pattern.

- **FastAPI Engine (Producer):** Handles incoming web requests asynchronously. Fast tasks (like DB reads or simple RDKit passes) are handled directly and returned.
- **MongoDB Task Queue:** Instead of relying on external queues like Redis/Celery, the system uses MongoDB as an atomic job queue to track pending ML tasks.
- **Background ML Worker (Consumer):** A persistent Python process (`worker.py`) running in isolation. It perpetually polls the database for `PENDING` jobs, performs PyTorch inference, and writes results back as `COMPLETED`.
- **Request Lifecycle (for ML tasks):**
    1. Client posts to `/jobs/optimize`.
    2. FastAPI creates a `PENDING` ticket in Mongo and immediately returns the `job_id` (HTTP 202 Accepted).
    3. The client begins polling `/jobs/{job_id}`.
    4. The background worker atomically claims the job using MongoDB's `find_one_and_update` with a concurrency lock.
    5. The worker executes the VAE prediction and updates the job document with the results.
    6. The client fetches the populated `COMPLETED` job.

## 3. API Design

The backend is mapped via RESTful routers built with FastAPI.

- **Authentication (`/auth` router):**
    - `POST /register`: Registers an account using Argon2 hashing.
    - `POST /login`: Issues short-lived JWTs (Access & Refresh tokens).
    - `POST /refresh`: Issues a new JWT context.
- **Entities (`/projects`, `/molecules` routers):**
    - Standard `GET`, `POST`, `DELETE` structured functionally. Project IDs act as foreign keys inside `Molecule` documents.
- **Jobs (`/jobs` router):**
    - `POST /optimize`, `POST /generate/targeted`: Issues async jobs.
    - `GET /{job_id}`: Polls the output.
- **Utilities (in `main.py`):**
    - `POST /utils/analyze`: Synchronous RDKit ingestion to score ADMET and check Toxicity.
    - `POST /assistant/chat`: Proxies context and questions to the Groq API.
- **Design Choice Checklist:** Employs standard REST conventions, standard HTTP status codes (200, 202 for async, 204 for delete, 401/404/500 for errors), and automatic OpenAPI documentation.

## 4. Database & Data Management

**Type:** MongoDB (NoSQL) operated asynchronously via `Motor` (`AsyncIOMotorClient`).
**Schema Design (Pydantic wrapper):**

- **Users Collection:** Stores `username`, `email`, and `hashed_password`.
- **Projects Collection:** Owned by a `user_id`.
- **Molecules Collection:** References a `project_id` and `user_id`. Stores nested `properties`, `admet`, and `tox_alerts` dictionaries.
- **Jobs Collection:** Stores `task_type`, `params` (payload), `status` (PENDING/PROCESSING/COMPLETED/FAILED), and `result`/`error`.
  **Key Operations:**
- **Worker Claim Query:** Uses `$set: {status: "PROCESSING"}` combined with sorting by `created_at` in an atomic `find_one_and_update` to guarantee that concurrent workers won't pick up the exact same job.

## 5. Core Logic & Important Modules

- **`chem_utils.py`:** Heavy reliance on RDKit. Calculates TPSA, LogP, and custom heuristic algorithms to assign normalized (0-1) ADMET probabilities and drug-likeness fitness scores. Utilizes `FilterCatalog` for PAINS toxicity alerts.
- **`ml_executor.py`:** The primary bridge tying PyTorch to the web system. Uses stateful global singletons to ensure the `VAE` and `Predictor` weights are only loaded into GPU/CPU memory once. Handles latent space noise additions and targeted property loss logic.
- **`assistant.py`:** Parses molecule context into strings and sets strict system prompts instructing the LLM (Groq) to act as a medicinal chemist without hallucinating structural data.

## 6. Authentication & Security

- **Strategy:** JWT (JSON Web Tokens) encoded via `HS256`.
- **Implementation:** OAuth2 Password Bearer flow.
- **Data Security:** Passwords are never stored in plaintext; they are hashed via `Argon2` (via Passlib context defaults), which is resilient to GPU brute force and rainbow table attacks.
- **Authorization:** Endpoints are protected via dependency injection (`Depends(get_current_user)`). The dependency parses the JWT, extracts the subject, and validates the user exists before proceeding. Users are physically prevented from deleting projects they do not own by querying explicitly with `user_id` constraint filters.

## 7. Performance & Scalability

- **Current Performance Specs:** FastAPI allows async HTTP bindings, and `Motor` ensures the I/O to the database is non-blocking. The heavy PyTorch logic is fully isolated into the `worker`.
- **Bottlenecks:** The current worker polls via a `while True:` loop every 2-5 seconds. As traffic increases, multiple instances polling MongoDB will throttle the DB resources.
- **Scalability Strategy:** While it currently runs locally, the decoupled logic means the `backend API` can scale horizontally behind a load balancer, and the `worker.py` can scale independently on GPU-enabled instances.

## 8. Error Handling & Reliability

- **Web Layer:** Throws `HTTPException` explicitly for missing files, invalid object IDs, and unmatched credentials.
- **Worker Layer:** Wrap executions in a `try...except` block. Critical failures (e.g., PyTorch OOM or decoding failures) print a stack trace, but they correctly mutate the MongoDB job status to `FAILED` and populate the `error` field so the frontend doesn't poll forever.

## 9. Tech Stack

- **Python 3.9+ / FastAPI:** Unmatched development speed, native async, and automatic data validation (Pydantic).
- **Motor (Async MongoDB):** Perfect pairing with FastAPI, allowing scalable non-relational database storage fitting messy molecular property structures seamlessly.
- **Passlib & Jose:** Industry standards for identity and cryptography.

## 10. Future Improvements

- **Redis/Celery Transition:** Moving the task queue from MongoDB to Redis for superior queuing efficiency.
- **WebSockets:** Replace client-side polling on `/jobs/{id}` with WebSockets to stream results in real-time, reducing HTTP overhead.
- **Caching:** Caching unchanged RDKit outputs or AI outputs in Redis to save compute on identical SMILES strings.

---

## 11. Backend-Focused Q&A Preparation

**Q1: Why did you choose FastAPI over Flask or Django?**
*Answer:* We chose FastAPI primarily for its native asynchronous capabilities (`async/await`) out of the box. Since we are dealing with high I/O (database callbacks, external ML worker polling, external API calls to Groq), the ASGI architecture allows it to handle much higher concurrency than sync frameworks like Flask. The automatic Pydantic validation was also critical for strictly defining our complex JSON payloads.

**Q2: How does the backend prevent the Machine Learning computation from blocking the API?**
*Answer:* We implemented an Asynchronous Producer-Consumer architecture. When a user requests molecule generation, the API does NOT run PyTorch. It writes a `PENDING` job ticket into MongoDB and immediately returns an HTTP 202 to the user. A separate Python process (the `worker.py`) constantly polls the database, claims the job, runs the ML model, and writes back the result. The client just polls the database until the status is `COMPLETED`.

**Q3: Why use MongoDB instead of a Relational DB (SQL) for this project?**
*Answer:* Molecular properties are highly diverse. Admet scores, toxicity alerts, structural violations, and dynamically generated traits could change in structure continuously. A NoSQL store like MongoDB allows us to dump variable-length property dictionaries without needing rigidly defining 15 different columns or tables in SQL. It's also extremely fast for the atomic document updates required by our custom queue.

**Q4: How did you ensure thread-safety when multiple workers might poll the database simultaneously?**
*Answer:* We use MongoDB's `find_one_and_update` command. It is an atomic operator at the database level. When a worker queries for a `PENDING` job, the database locks that document, returns it, and simultaneously flips its status to `PROCESSING` within the exact same transaction. This guarantees that two worker instances will never claim the exact same ML task.

**Q5: What limits your current architecture's scalability?**
*Answer:* The primary limit is the polling approach for jobs. Both the worker and the frontend actively poll the database every few seconds (using continuous GET requests), which could bottleneck the database connection pool at high scale.

**Q6: How would you fix that bottleneck in production?**
*Answer:* I would migrate the job queue from MongoDB to a dedicated message broker like Redis or RabbitMQ using Celery. For the frontend, instead of polling via `setInterval`, I would establish a WebSocket connection so the backend can push an event *exactly* when the ML task finishes.

**Q7: Explain how your Authentication system works.**
*Answer:* We use JWT (JSON Web Tokens). When a user logs in, we verify their password using the Passlib Argon2 hasher. If correct, we sign a payload indicating their `user_id` using our secret key via the HS256 algorithm and return an Access Token (30 mins) and a Refresh Token (7 days). On every secured request, the frontend sends the token in the `Authorization: Bearer` header, which our API decodes to identify the user before completing the action.

**Q8: If a user tries to delete a project belonging to someone else, how is this stopped?**
*Answer:* Security is handled at the ORM layer. The deletion endpoint doesn't just delete by `project_id`. The query is specifically written as: `db.projects.delete_one({"_id": project_id, "user_id": current_user.id})`. Because the `current_user.id` is reliably extracted from the cryptographically signed JWT, a malicious user can never satisfy that query for someone else's project.

**Q9: What happens if the generated molecule is completely invalid or fails during PyTorch execution?**
*Answer:* We catch exceptions at the worker layer. If the VAE fails, or the `chem_utils.py` parser throws an error during 3D generation, the `try...except` block intercepts it. Instead of crashing the worker, it logs the stack trace and updates the job status in MongoDB to `FAILED`, storing the stringified error message. The frontend catches this failed status and renders a notification, rather than hanging indefinitely.

**Q10: Why are the PyTorch models initialized globally inside `ml_executor.py`?**
*Answer:* Deep learning models are incredibly heavy and slow to load from disk into memory/VRAM. We load the VAE and the MLP predictor once as global singletons during the worker startup phase. Subsequent optimizations or generations use those pre-loaded resources in `torch.no_grad()` execution mode, allowing near-instantaneous inference.

**Q11: RDKit evaluates many properties directly. Why does the backend use simulated sigmoid/formulas for ADMET scores instead of just letting RDKit calculate ADMET?**
*Answer:* RDKit natively calculates *raw properties* (like TPSA and LogP) but does not provide pure ADMET predictive algorithms out of the gate (like Human Intestinal Absorption or Blood-Brain Barrier probabilities). We wrapped mapping functions around RDKit's raw outputs (using bounding boxes and exp transformations based on established pharmacokinetic literature) to provide a 0.0 to 1.0 probability score for the frontend UI.

**Q12: Is your password hashing secure? Why Argon2 over MD5 or SHA-256?**
*Answer:* Yes. MD5 and SHA-256 are fast generic hashers—that's a weakness for passwords because attackers can use GPUs to calculate billions of guesses per second. Argon2 (defaulted within our Passlib config) is intentionally slow, memory-hard, and saltable, rendering hardware-based brute-force attacks computationally impossible.

**Q13: How is the external AI chat functionality managed safely without exposing the API keys?**
*Answer:* The frontend sends the query and the literal 'molecule context' to our FastAPI `/assistant/chat` endpoint. The backend handles constructing the prompt and securely injecting the `GROQ_API_KEY` loaded from our protected `.env` server variables. The frontend never sees or interacts securely with the Groq service directly.

**Q14: Explain the importance of Pydantic in your project.**
*Answer:* We use Pydantic to strictly type incoming and outgoing API schemas (e.g., `MoleculeCreate` or `JobResponse`). FastAPI uses Pydantic to automatically parse incoming JSON, validate it against rules (like enforcing that an email actually looks like an email using `EmailStr`), and throws a clean 422 HTTP Unprocessable Entity error if the frontend ships incorrect types, saving us from writing hundreds of lines of boilerplate checking logic.

**Q15: If the server reboots, what happens to pending jobs?**
*Answer:* Since the Job Queue is managed transactionally in a persistent MongoDB disk collection (not in volatile memory), an API restart just means a temporary pause. The isolated worker process can still access MongoDB, and once the worker boots back up, it will resume claiming the longest-waiting `PENDING` jobs perfectly in order.